

PRÁCTICA 2 : Monitores**Productores-consumidores simple LIFO**

***Compilado con g++ -std=c++11 -pthread -o ./EXE/prodcons_su_LIFO.exe
prodcons_su_LIFO.cpp scd.cpp.***

En esta versión del problema de productor-consumidor, implementamos una solución LIFO, lo que significa que el último dato producido es el primero en ser consumido. Aquí hay una descripción de cómo funciona esta solución:

1. Utilizamos un búfer de tamaño fijo, donde se almacenan los datos producidos por los productores y consumidos por los consumidores.
2. Controlamos la posición de la primera celda libre en el búfer, que se inicializa en 0.
3. Cuando un hilo consumidor intenta leer un dato, verifica si el búfer está vacío. Si la primera celda libre es 0 (es decir, no hay elementos en el búfer), el hilo consumidor espera en la cola condición "ocupadas".
4. Cuando se retira un dato del búfer, se decrementa la variable que controla la primera celda libre, lo que permite que el siguiente dato producido se almacene en la posición anterior.
5. Luego de consumir un dato, se señala al hilo productor que hay un hueco libre en el búfer. Incrementamos la variable que controla la cantidad de celdas libres y señalamos a la cola condición "libres" para despertar a cualquier hilo productor que esté esperando.
6. Cuando un hilo productor intenta almacenar un dato en el búfer, verifica si el búfer está lleno. Si la primera celda libre es igual al tamaño del búfer (en este caso, 10), el hilo productor espera en la cola condición "libres".
7. Cuando se almacena un dato en el búfer, se incrementa la variable que controla la primera celda libre, permitiendo que el siguiente dato se almacene en la siguiente posición disponible.
8. Después de almacenar un dato, se señala al hilo consumidor que ya hay un dato disponible para ser consumido. Decrementamos la variable que controla la cantidad de celdas libres y señalamos la cola condición "ocupadas" para despertar a cualquier hilo consumidor que esté esperando.

Un ejemplo de salida es el siguiente:

```
mehdi@PCmehdi:~/SCD/PRÁCTICAS/PRÁCTICA 2$ g++ -std=c++11 -pthread -o ./EXE/prodcons_su_LIFO.exe prodcons_su_LIFO.cpp scd.cpp
mehdi@PCmehdi:~/SCD/PRÁCTICAS/PRÁCTICA 2$ ./EXE/prodcons_su_LIFO.exe
-----
Problema del productor-consumidor únicos (Monitor SU, buffer LIFO).
-----
hebra productora, produce 0
hebra consumidora, consume: 0
hebra productora, produce 1
hebra productora, produce 2
hebra consumidora, consume: 1
hebra productora, produce 3
hebra consumidora, consume: 2
hebra consumidora, consume: 3
hebra productora, produce 4
hebra consumidora, consume: 4
hebra productora, produce 5
hebra productora, produce 6
hebra consumidora, consume: 5
hebra consumidora, consume: 6
hebra productora, produce 7
hebra productora, produce 8
hebra productora, produce 9
hebra consumidora, consume: 7
hebra productora, produce 10
hebra consumidora, consume: 9
hebra productora, produce 11
hebra productora, produce 12
hebra consumidora, consume: 10
hebra productora, produce 13
hebra consumidora, consume: 12
hebra productora, produce 14
hebra consumidora, consume: 13
hebra consumidora, consume: 14
hebra consumidora, consume: 11
hebra consumidora, consume: 8
comprobando contadores ....
solución (aparentemente) correcta.
```

Productores-consumidores simple FIFO

Compilado con `g++ -std=c++11 -pthread -o ./EXE/prodcons_su_FIFO.exe prodcons_su_FIFO.cpp scd.cpp`.

En esta versión del problema de productor-consumidor, implementamos una solución FIFO, lo que significa que los datos se consumen en el mismo orden en que se produjeron. Aquí tienes una descripción de cómo funciona esta solución:

1. Utilizamos un búfer de tamaño fijo, que se representa como un arreglo, donde se almacenan los datos producidos por los productores y consumidos por los consumidores.
2. Controlamos la posición de la primera celda libre en el búfer, que se inicializa en 0 y de la última celda ocupada que también se inicializa a 0.
3. Cuando un hilo consumidor intenta leer un dato, verifica si el búfer está vacío. Si el contador de ocupadas es 0 (es decir, no hay elementos en el búfer), el hilo consumidor espera en la cola condición "ocupadas".

4. Cuando se retira un dato del búfer, se comprueba que primera_ocupada no esté al final del buffer para no leer una posición no reservada. En ese caso se pone la variable que indica la primera ocupada en 0. En el caso de que no, se incrementa la variable. Luego incrementamos el contador de libres y decrementamos el contador de ocupadas.
5. Luego de consumir un dato, se señala al hilo productor que hay un hueco libre en el búfer. Señalamos a la cola condición "libres" para despertar a cualquier hilo productor que esté esperando.
6. Cuando un hilo productor intenta almacenar un dato en el búfer, verifica si el contador de libres es 0. En ese caso, el hilo productor espera en la cola condición "libres".
7. Cuando se almacena un dato en el búfer, se incrementa el contador de ocupadas y se decrementa el contador de libres. Para la variable primera_libre se hace teniendo en cuenta la verificación del paso 4.
8. Después de almacenar un dato, se señala al hilo consumidor que ya hay un dato disponible para ser consumido. Para ello señalamos la cola condición "ocupadas" para despertar a cualquier hilo consumidor que esté esperando.

La salida es la siguiente:

```
mehdi@PCmehdi:~/SCD/PRÁCTICAS/PRÁCTICA 2$ g++ -std=c++11 -pthread -o ./EXE/prodcons_su_FIFO.exe prodcons_su_FIFO.cpp scd.cpp
mehdi@PCmehdi:~/SCD/PRÁCTICAS/PRÁCTICA 2$ ./EXE/prodcons_su_FIFO.exe
-----
Problema del productor-consumidor únicos (Monitor SU, buffer FIFO).
-----
hebra productora, produce 0
hebra consumidora, consume: 0
hebra productora, produce 1
hebra consumidora, consume: 1
hebra productora, produce 2
hebra productora, produce 3
hebra consumidora, consume: 2
hebra consumidora, consume: 3

hebra productora, produce 4
hebra consumidora, consume: 4
hebra productora, produce 5
hebra consumidora, consume: 5
hebra productora, produce 6
hebra productora, produce 7
hebra consumidora, consume: 6
hebra productora, produce 8
hebra consumidora, consume: 7
hebra productora, produce 9
hebra consumidora, consume: 8
hebra productora, produce 10
hebra consumidora, consume: 9
hebra productora, produce 11
hebra consumidora, consume: 10
hebra productora, produce 12
hebra consumidora, consume: 11
hebra productora, produce 13
hebra consumidora, consume: 12
hebra consumidora, consume: 13
hebra productora, produce 14
hebra consumidora, consume: 14
comprobando contadores ....

solución (aparentemente) correcta.
```

Productores-consumidores múltiples LIFO-FIFO

Se han compilado de esta forma:

```
g++ -std=c++11 -pthread -o ./EXE/prodcons_mu_LIFO.exe prodcons_mu_LIFO.cpp  
scd.cpp
```

```
g++ -std=c++11 -pthread -o ./EXE/prodcons_mu_FIFO.exe prodcons_mu_FIFO.cpp  
scd.cpp
```

Es igual que el prod-cons simple, lo único que cambia es que ahora tenemos un vector en el que se almacena los productos producidos por cada hebra y los consumidos por cada una.

Una posible salida LIFO es:

```
mehdi@PCmehdi:~/SCD/PRÁCTICAS/PRÁCTICA 2$ g++ -std=c++11 -pthread -o ./EXE/prodcons_mu_LIFO.exe prodcons_mu_LIFO.cpp scd.cpp
mehdi@PCmehdi:~/SCD/PRÁCTICAS/PRÁCTICA 2$ ./EXE/prodcons_mu_LIFO.exe
-----
Problema del productor-consumidor múltiples (Monitor MU, buffer LIFO).
-----
hebra productora 1 produce 3
hebra productora 2 produce 6
hebra productora 0 produce 0
hebra productora 4 produce 12
hebra productora 1 produce 4
hebra productora 3 produce 9
      consumido: 6 por la hebra 2
      consumido: 3 por la hebra 0
hebra productora 3 produce 10
hebra productora 0 produce 1
      consumido: 0 por la hebra 1
hebra productora 4 produce 13
hebra productora 2 produce 7
      consumido: 4 por la hebra 0
      consumido: 1 por la hebra 1
hebra productora 4 produce 14
hebra productora 3 produce 11
      consumido: 9 por la hebra 2
hebra productora 1 produce 5
hebra productora 0 produce 2
      consumido: 7 por la hebra 0
      consumido: 13 por la hebra 1
hebra productora 2 produce 8
      consumido: 11 por la hebra 2
      consumido: 2 por la hebra 0
      consumido: 5 por la hebra 1
      consumido: 8 por la hebra 2
      consumido: 14 por la hebra 0
      consumido: 10 por la hebra 1
      consumido: 12 por la hebra 2
comprobando contadores ....
solución (aparentemente) correcta.
```

Una posible solución FIFO es:

```
mehdi@PCmehdi:~/SCD/PRÁCTICAS/PRÁCTICA 2$ g++ -std=c++11 -pthread -o ./EXE/prodcons_mu_FIFO.exe prodcons_mu_FIFO.cpp scd.cpp
mehdi@PCmehdi:~/SCD/PRÁCTICAS/PRÁCTICA 2$ ./EXE/prodcons_mu_FIFO.exe
-----
Problema del productor-consumidor múltiples (Monitor MU, buffer FIFO).
-----
hebra productora 0 produce 0
hebra productora 1 produce 3
hebra productora 3 produce 9
hebra productora 4 produce 12
hebra productora 2 produce 6
hebra productora 1 produce 4
consumido: 0 por la hebra 0
consumido: 9 por la hebra 2
hebra productora 0 produce 1
consumido: 3 por la hebra 1
hebra productora 4 produce 13
hebra productora 3 produce 10
consumido: 12 por la hebra 0
hebra productora 2 produce 7
consumido: 4 por la hebra 1
hebra productora 1 produce 5
consumido: 6 por la hebra 2
hebra productora 4 produce 14
consumido: 1 por la hebra 0
consumido: 13 por la hebra 1
hebra productora 0 produce 2
consumido: 10 por la hebra 2
hebra productora 3 produce 11
hebra productora 2 produce 8
consumido: 7 por la hebra 0
consumido: 5 por la hebra 1
consumido: 14 por la hebra 2
consumido: 8 por la hebra 2
consumido: 11 por la hebra 1
consumido: 2 por la hebra 0
comprobando contadores ....
solución (aparentemente) correcta.
```

Fumadores

Compilado con:

`g++ -std=c++11 -pthread -o ./EXE/fumadores.exe fumadores_mo.cpp scd.cpp`

En este ejercicio creamos una solución con monitor para el problema de los fumadores.

Conservamos de la anterior solución las hebras, la función de fumar etc. Solo cambian las funciones de las hebras, que se le pasan ahora el monitor y se llama a sus correspondientes métodos.

Para el monitor creamos 4 colas tipo condition, 1 para cada fumadores y otra para el estanquero; y un entero para saber que elemento está disponible, valiendo -1 si no hay nada(como al principio).

Para la función obtener ingrediente comprobamos si el mostrador no está vacío y el elemento en el es el de un fumador determinado. En caso contrario hace un wait. Una vez pasa este filtro coge el ingrediente y hace un signal al estanquero.

Para poner ingrediente asignamos a mostrador el ingrediente y hacemos un signal del fumador correspondiente.

Finalmente para esperar la recogida de ingredientes hacemos un wait al estanquero si el mostrador no está vacío.

Una posible salida es la siguientes:

```
mehdi@PCmehdi:~/SCD/PRÁCTICAS/PRAC1-2_MehdiIabouten/PRÁCTICA 2$ g++ -std=c++11 -pthread -o ./EXE/fumadores.exe fumadores_mo.cpp scd.cpp
mehdi@PCmehdi:~/SCD/PRÁCTICAS/PRAC1-2_MehdiIabouten/PRÁCTICA 2$ ./EXE/fumadores.exe
-----
Problema de los fumadores (Monitor SU).
-----
Estanquero : empieza a producir ingrediente (16 milisegundos)
Estanquero : termina de producir ingrediente 1
Fumador 1 : empieza a fumar (155 milisegundos)

Estanquero : empieza a producir ingrediente (86 milisegundos)
Estanquero : termina de producir ingrediente 0
Fumador 0 : empieza a fumar (97 milisegundos)

Estanquero : empieza a producir ingrediente (56 milisegundos)
Estanquero : termina de producir ingrediente 1
Fumador 1 : termina de fumar, comienza espera de ingrediente.

Estanquero : empieza a producir ingrediente (41 milisegundos)
Fumador 1 : empieza a fumar (36 milisegundos)
Fumador 0 : termina de fumar, comienza espera de ingrediente.
Fumador 1 : termina de fumar, comienza espera de ingrediente.
Estanquero : termina de producir ingrediente 1
Fumador 1 : empieza a fumar (76 milisegundos)

Estanquero : empieza a producir ingrediente (47 milisegundos)
Estanquero : termina de producir ingrediente 2
Fumador 2 : empieza a fumar (185 milisegundos)

Estanquero : empieza a producir ingrediente (34 milisegundos)
Fumador 1 : termina de fumar, comienza espera de ingrediente.
Estanquero : termina de producir ingrediente 0
Fumador 0 : empieza a fumar (91 milisegundos)

Estanquero : empieza a producir ingrediente (85 milisegundos)
Estanquero : termina de producir ingrediente 1
Fumador 1 : empieza a fumar (103 milisegundos)

Estanquero : empieza a producir ingrediente (56 milisegundos)
Fumador 0 : termina de fumar, comienza espera de ingrediente.
Estanquero : termina de producir ingrediente 2
Fumador 2 : termina de fumar, comienza espera de ingrediente.

Estanquero : empieza a producir ingrediente (42 milisegundos)
Fumador 2 : empieza a fumar (181 milisegundos)
Fumador 1 : termina de fumar, comienza espera de ingrediente.
Estanquero : termina de producir ingrediente 2
Fumador 2 : termina de fumar, comienza espera de ingrediente.

Estanquero : empieza a producir ingrediente (72 milisegundos)
Fumador 2 : empieza a fumar (64 milisegundos)
Fumador 2 : termina de fumar, comienza espera de ingrediente.
Estanquero : termina de producir ingrediente 1
Fumador 1 : empieza a fumar (46 milisegundos)
```

Lectores Escritores

Compilado con:

`g++ -std=c++11 -pthread -o ./EXE/lec_escr.exe lec_escr.cpp scd.cpp`

En esta actividad hacemos un monitor en el que un único escritor escribe y varios lectores leen. Para probarlo usamos una variable global que se va incrementando, fuera del monitor, en las funciones de las hebras, ya que como no se especifica nada lo he hecho así para probarlo.

He lanzado 2 hebras escritoras y 5 lectoras.

Una salida posible es :

```
mehdi@PCmehdi:~/SCD/PRÁCTICAS/PRÁCTICA 2$ g++ -std=c++11 -pthread -o ./EXE/lec_escr.exe lec_escr.cpp scd.cpp
mehdi@PCmehdi:~/SCD/PRÁCTICAS/PRÁCTICA 2$ ./EXE/lec_escr.exe
-----
Problema de lectores_escritores (Monitor SU).
-----
El escritor 1 está escribiendo.
El escritor 1 ha finalizado la escritura.
El lector 3 está leyendo.
El lector 0 está leyendo.
El lector 1 está leyendo.
El lector 4 está leyendo.
El lector 2 está leyendo.
El lector El lector 1 ha finalizado la lectura.
2 ha finalizado la lectura.
El lector 3 ha finalizado la lectura.
El lector 0 ha finalizado la lectura.
El lector 4 ha finalizado la lectura.
El escritor 0 está escribiendo.
El escritor 0 ha finalizado la escritura.
El lector 3 está leyendo.
El lector 2 está leyendo.
El lector 1 está leyendo.
El lector 4 está leyendo.
El lector 0 está leyendo.
El lector 2 ha finalizado la lectura.
El lector 3 ha finalizado la lectura.
El lector 1 ha finalizado la lectura.
El lector 4 ha finalizado la lectura.
El lector 0 ha finalizado la lectura.
El escritor 1 está escribiendo.
El escritor 1 ha finalizado la escritura.
El escritor 0 está escribiendo.
El escritor 0 ha finalizado la escritura.
El lector 3 está leyendo.
El lector 1 está leyendo.
El lector 2 está leyendo.
El lector 0 está leyendo.
El lector 4 está leyendo.
El lector 3 ha finalizado la lectura.
El lector 2 ha finalizado la lectura.
El lector 0 ha finalizado la lectura.
El lector 3 está leyendo.
El lector 1 ha finalizado la lectura.
El lector 4 ha finalizado la lectura.
El lector 0 está leyendo.
El lector 3 ha finalizado la lectura.
El lector 2 está leyendo.
El lector 0 ha finalizado la lectura.
El lector 1 está leyendo.
El lector 2 ha finalizado la lectura.
El lector 4 está leyendo.
El lector 1 ha finalizado la lectura.
El lector 3 está leyendo.
El lector 2 está leyendo.
El lector 4 ha finalizado la lectura.
```

PRÁCTICA 3 : Paso de mensajes

Productores-consumidores múltiples -MPI

Compilado con make pcm

Vamos a implementar el problema de productores-consumidores múltiples usando MPI.

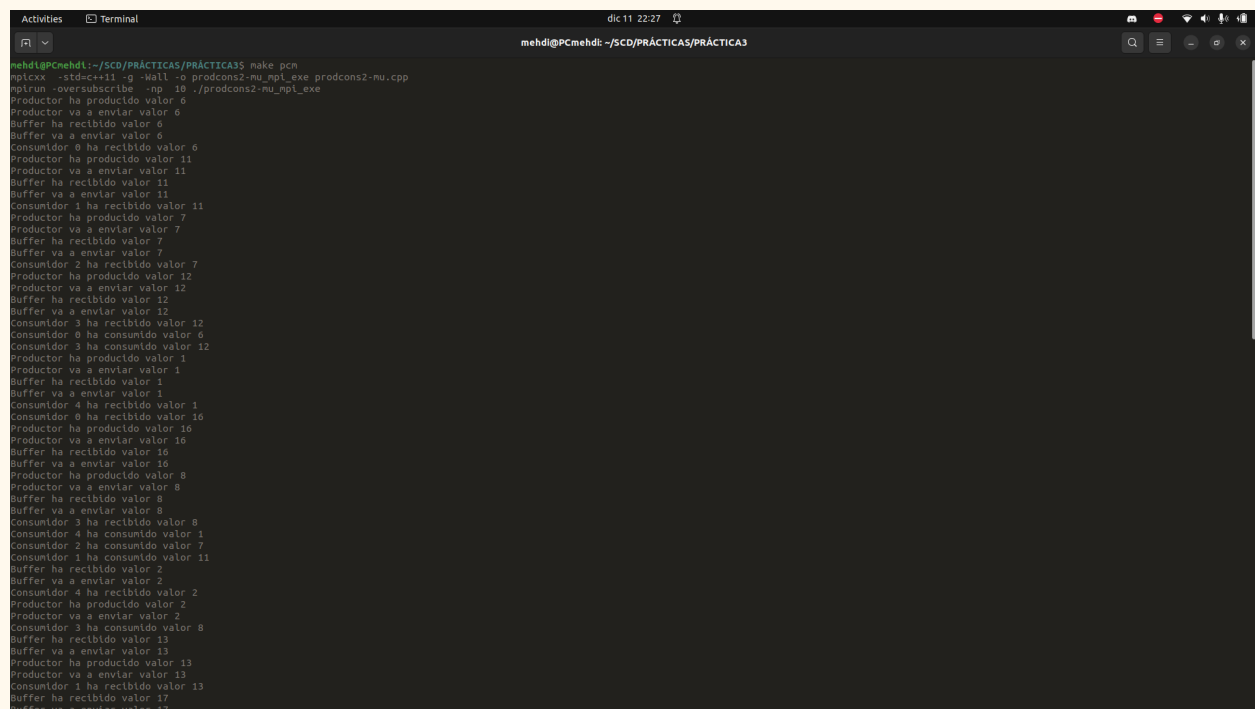
Para ello además de algunas modificaciones en las variables ya existentes añadimos nuevas variables como el `max_id_productor`, ...

Modificamos la función `producir` que ahora recibe el productor, con lo que calculamos el número de elementos a producir.

La función `consumir` también recibe el consumidor.

Finalmente creamos dos etiquetas distintas para diferenciar los mensajes de los productores y los consumidores. Modificamos la función `del buffer` para el uso de estas etiquetas.

Una posible salida es:



```
mehdi@PCmehdi:~/SCD/PRÁCTICAS/PRÁCTICA3$ make pcm
mpicxx -std=c++11 -g -Wall -o prodcons2-mu_mpi_exe prodcons2-mu.cpp
mpirun -oversubscribe -np 10 ./prodcons2-mu_mpi_exe
Productor ha producido valor 6
Productor va a enviar valor 6
Buffer ha recibido valor 6
Buffer va a enviar valor 6
Consumidor 0 ha recibido valor 6
Productor ha producido valor 11
Productor va a enviar valor 11
Buffer ha recibido valor 11
Buffer va a enviar valor 11
Consumidor 1 ha recibido valor 11
Productor ha producido valor 7
Productor va a enviar valor 7
Buffer ha recibido valor 7
Buffer va a enviar valor 7
Consumidor 2 ha recibido valor 7
Productor ha producido valor 12
Productor va a enviar valor 12
Buffer ha recibido valor 12
Buffer va a enviar valor 12
Consumidor 3 ha recibido valor 12
Consumidor 0 ha consumido valor 6
Consumidor 3 ha consumido valor 12
Productor ha producido valor 1
Productor va a enviar valor 1
Buffer ha recibido valor 1
Buffer va a enviar valor 1
Consumidor 4 ha recibido valor 1
Consumidor 0 ha recibido valor 10
Productor ha producido valor 10
Productor va a enviar valor 10
Buffer ha recibido valor 10
Buffer va a enviar valor 10
Productor ha producido valor 8
Productor va a enviar valor 8
Buffer ha recibido valor 8
Buffer va a enviar valor 8
Consumidor 3 ha recibido valor 8
Consumidor 4 ha consumido valor 1
Consumidor 2 ha consumido valor 7
Consumidor 1 ha consumido valor 11
Buffer ha recibido valor 2
Buffer va a enviar valor 2
Consumidor 4 ha recibido valor 2
Productor ha producido valor 2
Productor va a enviar valor 2
Consumidor 3 ha consumido valor 8
Buffer ha recibido valor 13
Buffer va a enviar valor 13
Productor ha producido valor 13
Productor va a enviar valor 13
Consumidor 1 ha recibido valor 13
Buffer ha recibido valor 17
Buffer va a enviar valor 17
```


Filósofos con interbloqueo

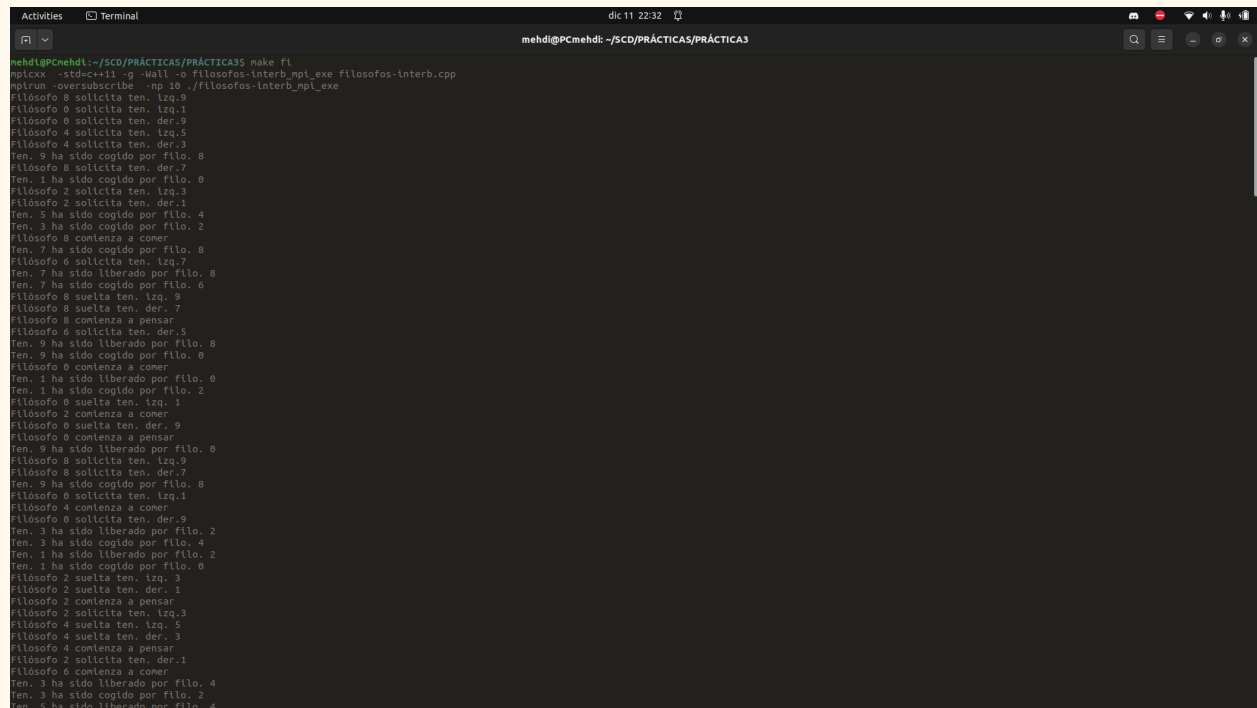
Compilado con make fi

Vamos a implementar el problema de filósofos con interbloqueo usando MPI.

Añadimos los envíos síncronos seguros y los recibimientos síncronos seguros.

Esta solución podría llegar a tener interbloqueo, ya que si por ejemplo todos cogen el de la izquierda a la vez nunca va a estar libre el de su derecha.

Una posible salida es:



```
mehdi@PCmehdi:~/SCD/PRÁCTICAS/PRÁCTICA5$ make fi
mpicxx -std=c++11 -g -Wall -o filosofos-interb_mpi_exe filosofos-interb.cpp
mpirun -oversubscribe -np 10 ./filosofos-interb_mpi_exe
Filósofo 8 solicita ten. izq.9
Filósofo 0 solicita ten. izq.1
Filósofo 6 solicita ten. der.9
Filósofo 4 solicita ten. izq.5
Filósofo 4 solicita ten. der.3
Ten. 9 ha sido cogido por filo. 8
Filósofo 8 solicita ten. der.7
Ten. 1 ha sido cogido por filo. 0
Filósofo 2 solicita ten. izq.3
Filósofo 2 solicita ten. der.1
Ten. 5 ha sido cogido por filo. 4
Ten. 3 ha sido cogido por filo. 2
Filósofo 8 comienza a comer
Ten. 7 ha sido cogido por filo. 8
Filósofo 6 solicita ten. izq.7
Ten. 7 ha sido liberado por filo. 8
Ten. 7 ha sido cogido por filo. 6
Filósofo 8 suelta ten. izq. 9
Filósofo 8 suelta ten. der. 7
Filósofo 8 comienza a pensar
Filósofo 6 solicita ten. der.5
Ten. 9 ha sido liberado por filo. 8
Ten. 9 ha sido cogido por filo. 0
Filósofo 0 comienza a comer
Ten. 1 ha sido liberado por filo. 0
Ten. 1 ha sido cogido por filo. 2
Filósofo 0 suelta ten. izq. 1
Filósofo 2 comienza a comer
Filósofo 0 suelta ten. der. 9
Filósofo 0 comienza a pensar
Ten. 9 ha sido liberado por filo. 0
Filósofo 8 solicita ten. izq.9
Filósofo 8 solicita ten. der.7
Ten. 9 ha sido cogido por filo. 8
Filósofo 0 solicita ten. izq.1
Filósofo 4 comienza a comer
Filósofo 6 solicita ten. der.9
Ten. 3 ha sido liberado por filo. 2
Ten. 3 ha sido cogido por filo. 4
Ten. 1 ha sido liberado por filo. 2
Ten. 1 ha sido cogido por filo. 0
Filósofo 2 suelta ten. izq. 3
Filósofo 2 suelta ten. der. 1
Filósofo 2 comienza a pensar
Filósofo 2 solicita ten. izq.3
Filósofo 4 suelta ten. izq. 5
Filósofo 4 suelta ten. der. 3
Filósofo 4 comienza a pensar
Filósofo 2 solicita ten. der.1
Filósofo 0 comienza a comer
Ten. 3 ha sido liberado por filo. 4
Ten. 3 ha sido cogido por filo. 2
Ten. 5 ha sido liberado por filo. 4
```

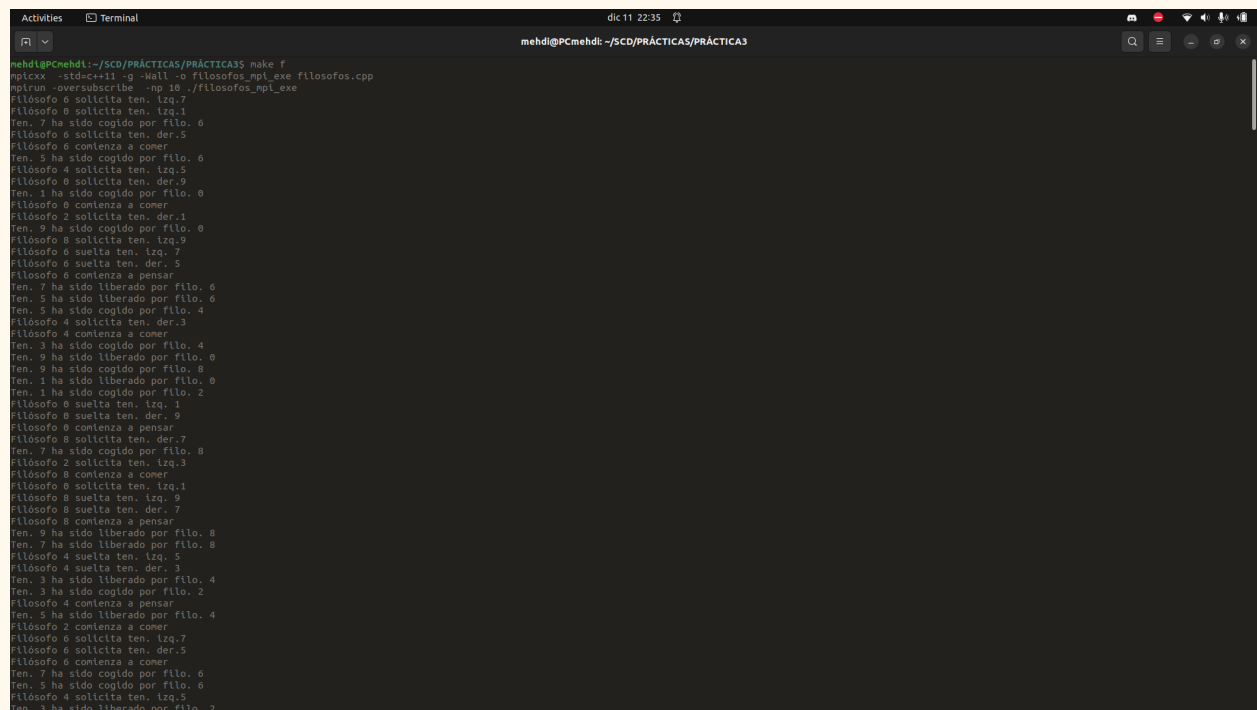
Filósofos sin interbloqueo

Compilado con make f

Vamos a implementar el problema de filósofos sin interbloqueo usando MPI.

Para solucionar el problema mencionado anteriormente, hacemos que uno de todos los filósofos coja primero el tenedor derecho antes del izquierdo a diferencia de los demás. Así evitamos que se puedan estancar todos si todos solicitan el izquierdo.

Una posible salida es:



```
mehdi@PCmehdi:~/SCD/PRÁCTICAS/PRÁCTICA3$ make f
mpicxx -std=c++11 -g -Wall -o filosofos_mpi_exe filosofos.cpp
mpirun -oversubscribe -np 10 ./filosofos_mpi_exe
Filósofo 0 solicita ten. izq.7
Ten. 7 ha sido cogido por filo. 0
Filósofo 0 solicita ten. der.5
Filósofo 0 comienza a comer
Ten. 5 ha sido cogido por filo. 0
Filósofo 4 solicita ten. izq.5
Filósofo 0 solicita ten. der.9
Ten. 1 ha sido cogido por filo. 0
Filósofo 0 comienza a comer
Filósofo 2 solicita ten. der.1
Ten. 9 ha sido cogido por filo. 0
Filósofo 8 solicita ten. izq.9
Filósofo 0 suelta ten. izq.7
Filósofo 0 suelta ten. der.5
Filósofo 0 comienza a pensar
Ten. 7 ha sido liberado por filo. 0
Ten. 5 ha sido liberado por filo. 0
Ten. 5 ha sido cogido por filo. 4
Filósofo 4 solicita ten. der.3
Filósofo 4 comienza a comer
Ten. 3 ha sido cogido por filo. 4
Ten. 9 ha sido liberado por filo. 0
Ten. 9 ha sido cogido por filo. 8
Ten. 1 ha sido liberado por filo. 0
Ten. 1 ha sido cogido por filo. 2
Filósofo 0 suelta ten. izq.1
Filósofo 0 suelta ten. der.9
Filósofo 0 comienza a pensar
Filósofo 8 solicita ten. der.7
Ten. 7 ha sido cogido por filo. 8
Filósofo 2 solicita ten. izq.3
Filósofo 8 comienza a comer
Filósofo 0 solicita ten. izq.1
Filósofo 8 suelta ten. izq.9
Filósofo 8 suelta ten. der.7
Filósofo 8 comienza a pensar
Ten. 9 ha sido liberado por filo. 8
Ten. 7 ha sido liberado por filo. 8
Filósofo 4 suelta ten. izq.5
Filósofo 4 suelta ten. der.3
Ten. 3 ha sido liberado por filo. 4
Ten. 3 ha sido cogido por filo. 2
Filósofo 4 comienza a pensar
Ten. 5 ha sido liberado por filo. 4
Filósofo 2 comienza a comer
Filósofo 6 solicita ten. izq.7
Filósofo 6 solicita ten. der.5
Filósofo 6 comienza a comer
Ten. 7 ha sido cogido por filo. 6
Ten. 5 ha sido cogido por filo. 6
Filósofo 4 solicita ten. izq.5
Ten. 3 ha sido liberado por filo. 2
```

Filósofos con camarero

Compilado con make fc

Vamos a implementar el problema de filósofos con camarero usando MPI.

Para este problema incorporamos un nuevo proceso, el camarero, el cual se encarga de permitir a los filósofos sentarse y levantarse. Solo se permite que haya cuatro personas sentadas simultáneamente. En caso de alcanzar este límite, el camarero solo autorizará que se levanten los filósofos, mediante el uso de una etiqueta. En el caso de que no haya ninguno sentado, solo se permitirá levantarse.

Una posible salida es:

```
mehdi@PCmehdi:~/SCD/PRÁCTICAS/PRÁCTICA3$ make fc
mpicxx -std=c++11 -g -Wall -o filosofos-can_mpi_exe filosofos-can.cpp
mpirun -oversubscribe -np 11 ./filosofos-can_mpi_exe
Filósofo 4 solicita sentarse.
Filósofo 6 solicita sentarse.
Filósofo 4 solicita ten. izq.5
Filósofo 0 solicita sentarse.
Filósofo 2 solicita sentarse.
Filósofo 4 se sienta.
Filósofo 2 se sienta.
Filósofo 6 se sienta.
Filósofo 0 se sienta.
Filósofo 2 solicita ten. izq.3
Filósofo 4 solicita ten. der.3
Filósofo 4 comienza a comer.
Ten. 5 ha sido cogido por filo. 4
Filósofo 6 solicita ten. izq.7
Filósofo 6 solicita ten. der.5
Filósofo 0 solicita ten. izq.1
Filósofo 0 solicita ten. der.9
Filósofo 0 comienza a comer
Filósofo 0 comienza a comer
Ten. 3 ha sido cogido por filo. 4
Ten. 7 ha sido cogido por filo. 6
Filósofo 8 solicita sentarse.
Ten. 9 ha sido cogido por filo. 0
Ten. 1 ha sido cogido por filo. 0
Filósofo 4 suelta ten. izq. 5
Filósofo 4 se levanta.
Filósofo 8 se sienta.
Filósofo 4 suelta ten. der. 3
Filósofo 4 se levanta. 3
Filósofo 4 comienza a pensar
Filósofo 8 solicita ten. izq.9
Filósofo 2 solicita ten. der.1
Filósofo 6 comienza a comer
Ten. 5 ha sido liberado por filo. 4
Ten. 5 ha sido cogido por filo. 6
Ten. 3 ha sido liberado por filo. 4
Ten. 3 ha sido cogido por filo. 2
Filósofo 6 se levanta.
Filósofo 6 suelta ten. izq. 7
Filósofo 6 suelta ten. der. 5
Filósofo 6 se levanta. 5
Filósofo 6 comienza a pensar
Ten. 5 ha sido liberado por filo. 6
Ten. 7 ha sido liberado por filo. 6
Filósofo 4 se sienta.
Ten. 5 ha sido cogido por filo. 4
Filósofo 4 solicita sentarse.
Filósofo 4 solicita ten. izq.5
Filósofo 4 solicita ten. der.3
Filósofo 0 suelta ten. izq. 1
Filósofo 0 suelta ten. der. 9
Filósofo 0 se levanta. 9
Filósofo 0 comienza a pensar
Ten. 9 ha sido liberado por filo. 0
```